DSPy-shaped model-backed programs

An optional, prepared feature set for Scale (D42)

Status

Prepared, optional, not applied. This note is the whole implementation for now: no source, no dependency, and no new word. The existing language, kernel, image, evaluator, scheduler, and verification pipeline are unchanged. A domain that never opts in is bit-for-bit unaffected.

What DSPy is

DSPy is a Python framework for building programs around language models. Its central claim is that you should not hand-write prompts. Instead you declare **what goes in and what comes out**, give a handful of examples and a scoring function, and the framework searches for the prompt that makes a specific model produce correct output.

Its architecture is four small pieces:

1. **Signature** — a typed declaration of the task: input fields, output fields, and a short natural-language instruction. In Python this is a Pydantic class with a docstring; constraints such as `ge=0`, `min_length=3`, and `Literal[...]` are stringified into the prompt and checked when the model's text is parsed back.
2. **`Predict`** — the one primitive that actually touches the language model. It formats the signature into a prompt, calls the model, and parses the reply back into typed fields. Higher-level modules (`ChainOfThought`, `ReAct`) are orchestration over one or more `Predict` calls.
3. **Adapter** — the pluggable layer that turns a signature plus examples into the model's prompt format, and parses the response. The lifecycle is fixed: preprocess, format, call, postprocess, parse.
4. **Optimizer** — a search loop that walks the program, finds every `Predict`, runs it on a training set, and rewrites exactly two things per predictor: the instruction string and the few-shot examples. It keeps candidates that score well on a validation set. GEPA is a reflective optimizer used with DSPy: it sends execution traces to a model and asks it to propose better instructions.

Two properties matter for the fit. First, the signature is a *runtime* annotation in Python, so a program must extract it before it can be used. Second, the constraints in a signature are *not enforced during generation*: the model can emit anything, and the constraint is checked afterwards, when parsing fails.

The proposal in Scale terms

Treat the DSPy proposal as an **additional, optional feature set**, not as a replacement for Scale's specification, verification, or concurrency tracks. The one rule that keeps it compatible is this: the signature is a projection of the existing `.scale` contract, not a second source of truth.

DSPy concept	Scale fit
Signature	A native value: named input and output fields with value sets (D12, D31, D36), an instruction, and the verification lattice (D25) projected from the same `.scale` document. The parsed sentence already names the subject, its fields, and the verb.

DSPy concept	Scale fit
`Predict`	One edge word, the only model-touching primitive. It returns a `Result` with a named refusal, never an exception.
Adapter	A host port at the edge, with the fixed five-step lifecycle. It parses the reply into the signature type; parsing is the boundary, not a post-hoc check.
Optimizer	A host-side search driver over instruction text and examples. It never edits the `.scale` document and never mutates the program; its result is a new append-only artifact.
Metric	Two layers: a fast search metric (type check, value-set membership, property tests) and the slow acceptance gate (`Smt.discharge` plus `FixedSpecificationAcceptance.accept`). Only the slow gate can accept.
`compile`	Called **optimize** in Scale. DSPy's `compile` and Scale's compiler are different jobs; the name stays free for the emitter.

What must be preserved

1. `predict` stays the only model-touching primitive; everything else composes it.
2. The adapter stays pluggable, with the fixed five-step lifecycle.
3. A metric is required: no metric, no optimization.
4. The optimizer rewrites exactly two things per predictor: the instruction and the examples.
5. The contract, the strategy, and the types are developer-owned; the search may never change them.
6. The optimized result is a new artifact. Scale's append-only analog of DSPy's deep copy: the original program and the `.scale` document are untouched.

How it fits, layer by layer

Surface: the sentence is already a signature

The Prose sentence grammar already produces a parsed sentence with a subject, typed conditions, and a verb. That is the signature: the subject's fields are the inputs, the verb's result is the output, and the sentence itself is the instruction. A developer-facing signature value is therefore a projection of the document, and the child-facing surface does not change. The stack words remain the lab surface beneath it.

Image: the program and its transcript are data

A signature, a prediction, and an optimized artifact are image entries, not host records. The image is append-only (D4, D16), so optimizing a program returns a new entry and leaves the original intact. A model call is an effect, so a transcript (request and response) belongs in an optional record/replay cassette, not in the proof. Replay is what makes a search reproducible when the model is not.

Evaluator: one model-touching word at the edge

`predict` is a word that consumes a signature and its inputs and delegates to an injected adapter. The adapter is a host resource, so it is passed at the run boundary, like `executeWord`, and is not serialized into the image. A refusal is a named value: unavailable model, malformed reply,

schema mismatch, or an undecided constraint. The evaluator's existing rule — failures are values — covers the whole boundary.

The model call is *not* a kernel operation. The kernel keeps its stack, image, literals, and dispatcher; a domain that does not install an adapter never sees the word.

Verification: fast search, slow gate
The fast metric is a search signal. It parses the reply into the signature type, checks value-set membership, and runs the property tests over the finite lattice. It may call Hedgehog and the lattice generators; it does not call Z3 and it does not accept a program.

The slow metric is the existing acceptance gate. `Smt.discharge` answers `Proved`, `Falsified (model)`, `Unknown`, or `Unavailable`, and `FixedSpecificationAcceptance.accept` still requires the target proof, the witness corpus, an unchanged specification hash, and the differential check. A score from the fast metric is never a proof and never a pass. The same four-way honesty applies to the model: its output is untrusted input, parsed at the edge, and the verifier proves the program around it, not the model itself.

Optimizer: host-side, fixed-spec by construction
DSPy's optimizer rewrites instructions and examples. Scale's fixed-spec mode (D11) already says the specification is fixed and the spec-append path is unreachable by rule. The two fit together: the optimizer may change presentation and examples, never the contract. Its output is a new artifact with provenance (model, adapter, metric version, solver version), and the original program stays in the image. The specification hash pins the contract across the search.

Scheduler: do not block an actor on a model
A model call is external, blocking I/O. The cooperative scheduler must not park an actor on a network call. Run the adapter at the edge and feed the result back as a value; if a program composes several predictions, represent the outstanding requests as events or edge work, not as a blocked actor. A budget and a timeout are adapter concerns, reported as named refusals.

Synergies

1. **The parsed sentence is the signature.** No extractor is needed: the subject's fields, the conditions, and the verb are already typed data from the parser and the lattice.
2. **The verification lattice is the input schema.** `TypeLattice` already names fields, sorts, finite coordinates, and narrowing; the fast metric reads the same coordinate system as Z3 and Hedgehog.
3. **Hedgehog shrinking plus the repair bundle is the reflection signal.** Stock DSPy reflects on full traces. Scale shrinks a failure to a minimal counterexample and already packages the witness, the solver model, the spec hash, and the differential result. That bundle is a sharper, versioned interface for an optimizer to consume.
4. **`FixedSpecificationAcceptance.accept` is the slow metric.** The four-rule gate already exists. The optimizer does not need a new acceptance oracle.
5. **The event-stream gives record and replay.** The history/step model (D29) is the natural place to record a prediction, so a search can be replayed.
6. **Metaobjects give the signature a representation without a kernel case.** A signature can follow the D16/D17 pattern: a tagged image entry, a projection, and a protocol, not another explicit `executeWord` case.
7. **Fixed-spec mode bounds the search.** The optimizer is a presentation search, not a contract rewrite, so it cannot violate the append-only document or the spec hash.

Tensions and resolutions

1. **Constrained decoding is not universal.** Only the structural fragment of a signature can drive a decoding grammar: field names, booleans, and finite value sets. Predicate and constructor sorts are runtime predicates, not grammars; they are checked after parsing and can answer `Undecided`. Symbolic fields cannot be enumerated at all. Never promise a grammar for an arbitrary quotation.
2. **Models are nondeterministic; Scale's evaluation is deterministic.** A model call is an effect. Record it in a cassette and state the determinism property over the recording, or exclude the word from that property by name. Do not silently weaken the existing law.
3. **DSPy's `compile` rewrites; Scale's image does not.** The optimizer's output is a new artifact and a new image entry. The original program and the `.scale` document are immutable. This is the same discipline as the refinement loop: append, never rewrite.
4. **A metric is not a proof.** A high score only ranks candidates. The slow gate is what closes a class; `Unknown`, `Unavailable`, and an inconclusive run never read as a pass.
5. **SMT cannot reason about prompt text.** The verified theory stays the finite typed fragment (D25). The model's prose, the instruction, and the examples are outside the solver; the host program and its typed post-processing are inside it.
6. **Privacy and append-only transcripts.** A transcript can contain user data, and an append-only image cannot delete it. Keep transcripts in an optional, redacted cassette, off by default, rather than in the image. Never put secrets in a signature or an example.
7. **The signature must not become a second source of truth.** It is a projection of the `.scale` document and the lattice, like the emitter and the SMT obligation. If it were separately editable, the contract and the program could drift, which is exactly the seam D25 exists to close.
8. **The optimizer is host code.** Keep it out of the kernel: it is a hexagonal adapter/driver, not a new semantic primitive. The core project does not take a model SDK as a dependency.
9. **Cost and latency.** A search loop multiplies model calls. Cache predictions, bound the candidate count, and keep the fast metric cheap; a wrong search is a slow search, not a proof failure.

What would have to change

Nothing fundamental. Scale can exist exactly as it does today without this feature set. Promotion would add an optional vertical slice:

1. a signature value or metaobject that projects to and from the `.scale` document and the lattice;
2. a `predict` port with named refusals, a budget, and a timeout;
3. a record/replay cassette for model calls;
4. a host-side optimizer that emits a new artifact and never edits the specification;
5. fast-metric and slow-gate tests, including a differential check over a recorded transcript.

The core image, evaluator, scheduler, error semantics, and verification pipeline would not change. A domain opts in by installing an adapter and declaring a signature; a domain that does not is unaffected.

Non-goals

1. Training or fine-tuning a model.
2. Proving the model's behaviour, the prompt's faithfulness, or the instruction's meaning.

3. A reflective evaluator or a general prompt language.
4. Replacing `.scale`, the refiner, or `FixedSpecificationAcceptance.accept`.
5. Network access in the kernel.
6. Treating provider-specific constrained decoding as a guarantee.
7. Storing secrets or unredacted transcripts by default.
8. Making the fast metric an acceptance oracle.

Promotion criteria

1. A signature that is a projection of the document and the lattice, not a second source of truth.
2. One model-touching word with named refusals and a record/replay cassette.
3. A fast metric that is search-only, and a slow metric that reuses `Smt.discharge` and `FixedSpecificationAcceptance.accept`.
4. An optimizer that never edits the `.scale` document and emits a new append-only artifact.
5. Differential tests over a recorded transcript; `Proved`, `Falsified`, `Unknown`, and `Unavailable` reported honestly.
6. No change to a domain that does not opt in.

One-line version

Port the DSPy shape so the signature is the `.scale` contract's projection; keep `predict` the only model-touching word; run the optimizer host-side over instructions and examples only; search on the fast metric; accept only through the existing proof-or-witness gate; record model calls for replay; treat the model as the one thing you cannot prove.